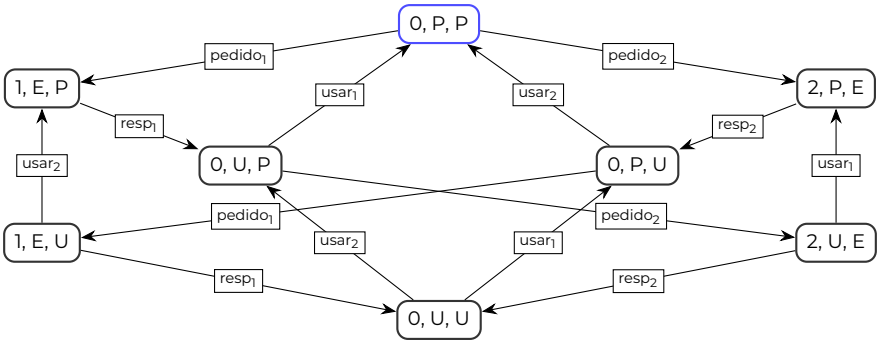
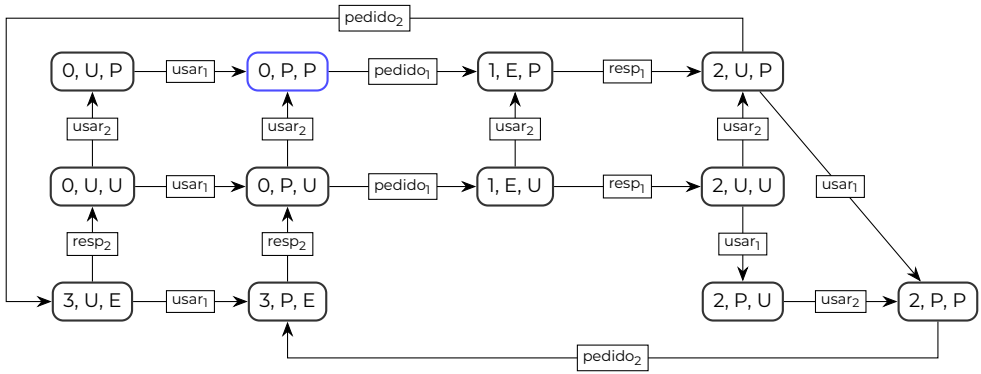
Versión A: el diagrama



Versión B: el diagrama



Las proposiciones sobre los estados

Cada proposición es una condición sobre una componente de la terna.

Proposición	**Versión A**	**Versión B**
$pide_i$	cliente *i* en P	cliente *i* en P
$espera_i$	cliente *i* en E	cliente *i* en E
usa_i	cliente *i* en U	cliente *i* en U
$atiende_1$	servidor en 1	servidor en 1
$atiende_2$	servidor en 2	servidor en **3**

- Recordemos: la traza de una ejecución es la sucesión de estados que visita. El LTS satisface φ si la satisfacen las trazas de **todas** sus ejecuciones.
- Como no hay estados sin salida, todas las ejecuciones son infinitas.

Recordemos los operadores

Fórmula	**Vale en la posición *i* si...**
$\Box\varphi$	φ vale en *i* y en todas las siguientes
$\Diamond\varphi$	φ vale en *i* o en alguna siguiente
$\bigcirc\varphi$	φ vale en $i+1$
φ **U** ψ	ψ vale en algún $j \geq i$, y φ vale en todas las anteriores a *j*
φ **W** ψ	como **U**, pero si ψ no llega nunca, alcanza con $\Box\varphi$

φ **W** $\psi \equiv (\varphi$ **U** $\psi) \vee \Box\varphi$. Con lo cual **U exige** que ψ llegue, y **W** no.

Recordemos: safety y liveness

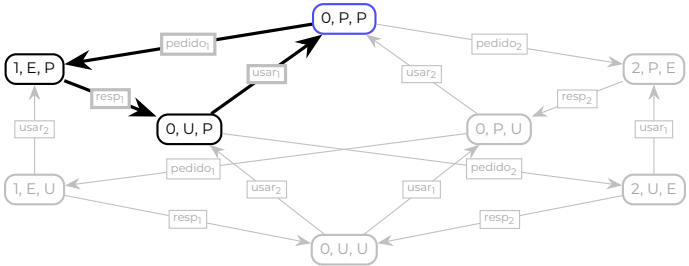
Safety

Algo malo nunca pasa. Si una traza la viola, hay un **momento finito** en que ya se violó, y nada de lo que venga después lo arregla.

Liveness

Algo bueno termina pasando. Ningún prefijo finito la viola, porque lo bueno siempre puede llegar más adelante.

1. $\square\lozenge usa_2$ en A: no vale

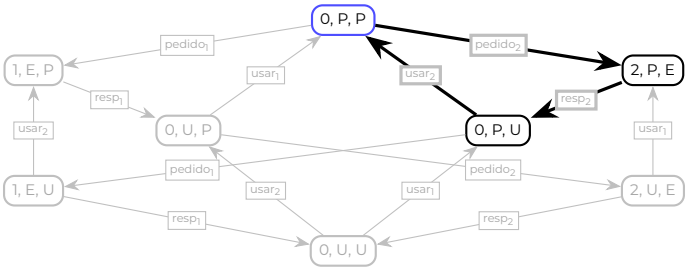


El `select` puede elegir siempre al cliente 1: $\big((0,P,P)\,(1,E,P)\,(0,U,P)\big)^{\omega}$. La vuelta nunca pasa por un estado con el cliente 2 en U, que se queda en P para siempre.

Pregunta

¿Es válido este lazo bajo fair choice?

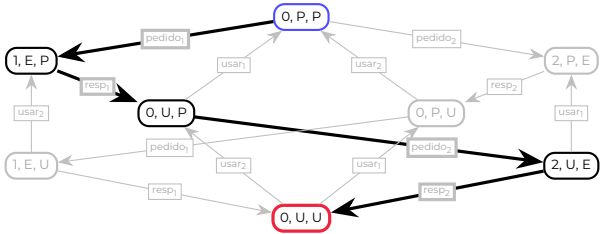
2. $\Box(pide_1 \rightarrow \Diamond espera_1)$



En A no vale. Es lo mismo al revés. El `select` elige siempre al cliente 2, $((0, P, P)\ (2, P, E)\ (0, P, U))^{\omega}$, y el cliente 1 se queda en P.

- **Bajo fair choice vale**, por el mismo argumento: hay una sola componente terminal y tiene estados con el cliente 1 en E.

3. $\Box\neg(usa_1 \wedge usa_2)$: no vale en ninguna



En palabras: nunca hay un estado en el que los dos clientes estén usando una respuesta a la vez.
En A, un prefijo finito: (0, P, P) (1, E, P) (0, U, P) (2, U, E) (0, U, U). En B, la misma secuencia de acciones también llega a (0, U, U).

- El servidor no espera a que un cliente termine de usar para atender al otro.
- **Fair choice no cambia nada.** Es safety, el prefijo malo ya ocurrió, y se puede continuar de forma justa.

4. $\square(espera_1 \rightarrow \bigcirc(espera_1 \vee usa_1))$

En palabras: si el cliente 1 está esperando, en el estado siguiente sigue esperando o ya está usando. Nunca vuelve a P sin haber usado la respuesta.

Pregunta

¿$\bigcirc$ es “el próximo paso del cliente 1”?

No. $\bigcirc$ es el próximo paso **del sistema**. Puede ser un paso del servidor, del cliente 1 o del cliente 2.

- Es **safety**. Un contraejemplo sería un prefijo que termina en un paso que saca al cliente 1 de E hacia P.

4. Vale en las dos

Busquemos un paso que saque al cliente 1 de E hacia algo distinto de U. Usemos la regla de composición:

- Si la acción no está en el alfabeto de C_1 (pedido_2, resp_2, usar_2), el cliente 1 no se mueve. Sigue en E.
- Si está en el alfabeto de C_1, tiene que ofrecerla C_1. Desde E, C_1 solo ofrece resp_1, que lo lleva a U.

No usamos nada del servidor, con lo cual el argumento vale igual en A y en B. Como no hay estados sin salida, siempre hay un estado siguiente y $\bigcirc$ tiene sentido.

Es la misma idea que la propiedad 2 con **W**. La fórmula equivale a

$$\Box(\mathit{espera}_1 \rightarrow (\mathit{espera}_1 \ \mathbf{W} \ \mathit{usa}_1))$$

es decir, desde que espera, sigue esperando hasta usar (o para siempre).